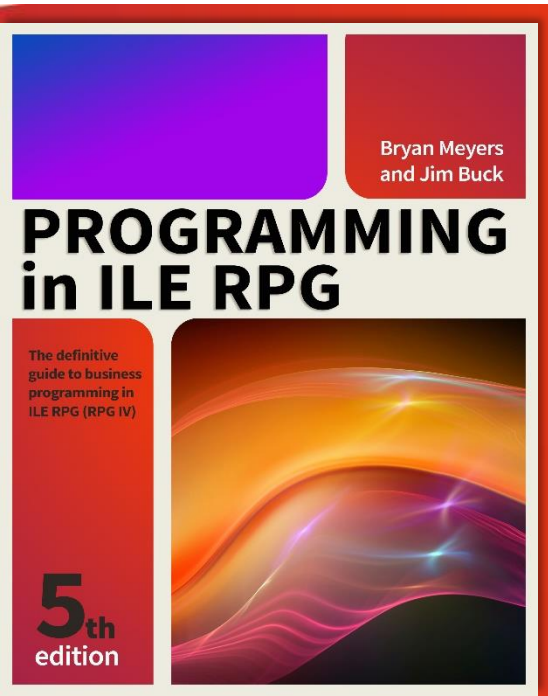
Chapter 5: Controlling Program Workflow



Presentation © Copyright 2015, Enskill.com

Focus

- Focus on Slides 1 through 21.
- Also watch the video following this Powerpoint
- Notes about the video
  - You will see some older style code in this video.
  - Ignore all references to page numbers

Structured Design

- Three basic logic structures
  - Sequence
    - Program processes statements sequentially from top to bottom
    - Default behavior
  - Selection
    - Alternative processing path depending upon runtime conditions
  - Iteration (looping)
    - Repeat instructions depending upon runtime conditions
      - DOW NOT %EOF
- Each structure has single entry point and single exit point
  - Maintain tight control over program flow

Structured Operations

- Selection operations
  - If
    - Else
    - Elseif (Else If)
  - Select (Conditionally select operations)
    - When (When true)
    - Other (Otherwise)
- Iteration operations
  - Dow (Do While)
  - Dou (Do Until)
  - For

Relational Comparisons

- Selection and iteration operations test relational condition to set course of action
- Relational symbols
  - > (Greater than)
  - >= (Greater than or equal to)
  - = (Equal to)
  - <= (Less than or equal to)
  - < (Less than)
  - <> (Not equal to)
- Comparisons use EBCDIC collating sequence
- Cannot directly compare incompatible data types

Logical Comparisons

- Structured operations can compare indicators with *On or *Off
  - Reserved indicators are prefixed with *In
  - Need not prefix named indicators

```
Dcl-s Endofpage Ind;

If Endofpage;     // same as If Endofpage = *On;
  …
Endif;

If Not *In99;     // same as If *In99 = *Off;
  …
Endif;
```

If

- If condition is true, program executes calculations in If block
  - Block ends with Endif
  - If relationship is not true, block is bypassed

```
If Age >= 65;
  Count += 1;
Endif;

If (Credlimit – Amtowed - Ordertot) >= 0;
  Approval = *On;
  Amtowed += Ordertot;
Endif;
```

If

- May test multiple conditions
  - And
    - Both conditions must be true
  - Or
    - Either condition (or both) must be true

```
If Age >= 65 And Status = 'R';
  Count += 1;
Endif;
```

```
If Age >= 65 Or Status = 'R';
  Count += 1;
Endif;
```

Else

- Else provides alternate processing block if condition is not true

```
If Hours <= 40;
  Totalpay = Hours * Payrate;
Else;
  Totalpay = 40 * Payrate + (Hours - 40) * Payrate * 1.5;
Endif;
```

Nested If Blocks

- Can nest If block inside other If blocks
  - With or without Else
- **Each If block requires Endif**

```
If Sales <= 5000;
  Rate = .005;
Else;
  If Sales <= 10000;
    Rate = .0075;
  Else;
    If Sales <= 20000;
      Rate = .01;
    Else;
      Rate = .015;
    Endif;
  Endif;
Endif;
```

Elseif (Else If)

- Elseif combines Else and If operations
  - Simplifies nested If blocks
- **Requires only one Endif**

```
If Sales <= 5000;
  Rate = .005;
Elseif Sales <= 10000;
  Rate = .0075;
Elseif Sales <= 20000;
  Rate = .01;
Else;
  Rate = .015;
Endif;
```

Select (Conditionally Select Operations)

- Select and Endsl operations selection group for processing
  - Simplify case logic
  - **May substitute for nest If blocks**

```
Select;
  When Sales <= 5000;
    Rate = .005;
  When Sales <= 10000;
    Rate = .0075;
  When Sales <= 20000;
    Rate = .01;
  Other;
    Rate = .015;
Endsl;
```

Select (Conditionally Select Operations)

- When operations test conditions inside Select group
  - Only first appropriate When block will execute
  - Other operation designates processing if all When conditions are false

```
Select;
  When Sales <= 5000;
    Rate = .005;
  When Sales <= 10000;
    Rate = .0075;
  When Sales <= 20000;
    Rate = .01;
  Other;
    Rate = .015;
Endsl;
```

Dow (Do While)

- Dow loop executes repeatedly as long as condition is true
  - Block ends with Enddo
- May form compound conditions with And/Or
- Processing inside loop must change condition state to avoid endless loop
- Leading decision loop
  - Tests condition before loop is executed
  - Will not execute loop if condition is initially false

```
Dow Number < 100;
  Number += 1;
  Sum += Number;
Enddo;
```

Dou (Do Until)

- Dou loop executes repeatedly until condition is true
  - Bock ends with Enddo
- May form compound conditions with And/Or
- Processing inside loop must change condition state to avoid endless loop
- Trailing decision loop
  - Tests condition after loop is executed at least once

```
Dou Number >= 100;
  Number += 1;
  Sum += Number;
Enddo;
```

For

- For repeats code block specific number of times
  - Block ends with Endfor
- Counter is automatically incremented after each iteration
  - Block repeats until maximum counter value is exceeded

```
For Number = 1 To 100;
  Sum += Number;
Endfor;
```

Loops and Early Exits

- Iter
  - Returns to Dou, Dow, For
- Leave
  - Branches to statement following end of block

```
Dou %Eof(Customers);
  Read Customers;
  Select;
    When %Eof;
      Leave;
    When Amountdue = 0;
      Iter;
    Other;
      Except Detail;
  Endsl;
Enddo;
```

File Processing with Dow and Dou

- Dow requires priming read before loop

```
Read Customers;

Dow Not %Eof(Customers);
  Count += 1;
  Write Detail;
  Read Customers;
Enddo;
```

- Dou requires early exit from loop at %Eof

```
Dou %Eof(Customers);
  Read Customers;

  If %Eof(Customers);
    Leave;
  Endif;

  Count += 1;
  Write Detail;
Enddo;
```

Defining and Using Subroutines

- Subroutine is named block of code executed within a program
  - Improves program organization, maintainability
  - Code subroutines after all other processing
  - Subroutines are not recursive
- Begsr (Begin Subroutine) operation marks start of subroutine
  - Endsr (End Subroutine) operation marks end
- Exsr (Execute Subroutine) operation invokes subroutine block
  - When subroutine reaches Endsr, control returns to next line after Exsr
- Leavesr (Leave Subroutine) operation provides early exit from subroutine

Defining and Using Subroutines

```
Exsr Calctax;    // Exsr causes control to drop to subroutine Calctax
// Control returns here when the subroutine finishes
.
.
.
// Subroutines appear at the end of the processing
Begsr Calctax;
  Fica =  Gross * .0765;
  Statetax = Gross * .045;

  If Gross > 5000;
    Fedtax = Gross * .31;
  Else;
    Fedtax = Gross * .25;
  Endif;

Endsr;
```

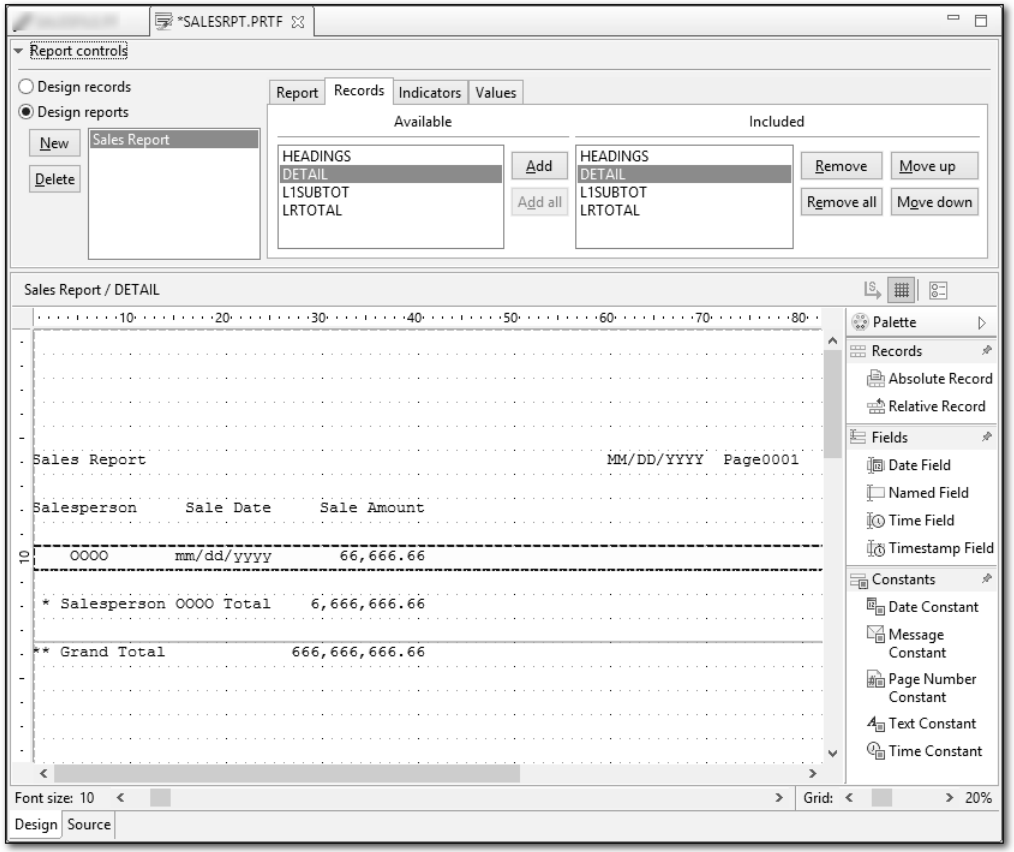
Using the *INZSR Subroutine

- *INZSR is a specially named subroutine that executes automatically when program starts
  - Provided initial processing for program
  - Exsr can also execute *INZSR

```
Begsr *Inzsr;
  Default = .0751;
  Action = 'A';
Endsr;
```

Control Break Logic – Glance through this section

- Used to program subtotals when control field value changes
  - Control break signals need for subtotal, special processing



Control Break Logic

- Program must declare variable to hold current value of control field
- For each record, program must check to see if “hold variable” has changed from previous record
- If value has changed, process subtotal before processing current record
  - Control break

Single Level Control Break Logic

- Read first record
- Assign value to control field hold area for first record
- While not end-of-file
  - If control field value changes, execute subtotal subroutine
    - Print subtotal
    - Add subtotal to grand total (“roll up”)
    - Zero out subtotal
    - Assign current record’s value to control field hold area
  - Execute detail subroutine for current record
    - Print detail line
    - Add current record’s total to subtotal
  - Read next record
- Execute termination subroutine
  - Print final subtotal
  - Print grand total
- End program

Single Level Control Break Sample

```
Dcl-f Salesfile Disk    Keyed;
Dcl-f Salesrpt  Printer Oflind(Endofpage);

Dcl-s Endofpage Ind Inz(*On);
Dcl-s L1Slspers     Like(Slsperson);

// ************************************************************
Read Salesfile;
L1Slspers = Slsperson;

Dow Not %Eof(Salesfile);
  If L1Slspers <> Slsperson;
    Exsr L1break;
  Endif;
  Exsr Detailproc;
  Read Salesfile;
Enddo;

Exsr Terminate;
*Inlr = *On;
Return;
```

(Continued...)

```
// ***************************************************
//  Subroutine Detailproc ... executed for each input record.
Begsr Detailproc;
  Exsr Overflow;
  Write Detail;
  L1Amount += Saleamount;
Endsr;


// ***************************************************
//  Subroutine L1Break ... done when salesperson changes
Begsr L1Break;
  Exsr Overflow;
  Write L1Subtot;
  LRAmount += L1Amount;
  L1Amount = 0;
  L1Slspers = Slsperson;
Endsr;


// ***************************************************
//  Subroutine Terminate ... done at end-of-file
Begsr Terminate;
  Exsr L1Break;
  Exsr Overflow;
  Write LRTotal;
Endsr;
```

(Continued...)

```
// ***************************************************************
//  Subroutine Overflow ... print headings when necessary
Begsr Overflow;

  If Endofpage;
    Write Headings;
    Endofpage = *Off;
  Endif;

Endsr;
```

Multiple Level Control Break Logic

- Program must declare variables to hold current value of each control field
  - Useful to name “hold variables” to indicate level
    - L1name, L2name, L3name, … (L1 is most minor level)
- For each record, program must check to see if any “hold variable” has changed from previous record
  - Check from major to minor order (L3, L2, L1)
- If any value has changed, process subtotals for any previous level breaks and for current level break before processing current record
  - Process in order from most minor to current level (L1, L2, L3)
  - When level break occurs, all minor levels below it also break

Multiple Level Control Break Logic

Main Routine

- Read first record
- Assign values to all control field hold areas for first record
- While not end-of-file
  - Check for control field value changes, in major to minor order
    - Process control break subroutine for most major break
  - Execute detail subroutine for current record
    - Print detail line
    - Add current record's total to most minor subtotal
  - Read next record
- Execute termination subroutine
  - Process most major control break subroutine
  - Print final subtotal
  - Print grand total
- End program

Each Control Break Subroutine

- Process previous level control break subroutine
- Print subtotal for current level control break
- Add current subtotal to next level subtotal ("roll up")
- Zero out current level subtotal
- Assign current record's values to control field hold areas

Multiple Level Control Break Sample

```
Dcl-f Salesfile Disk    Keyed;
Dcl-f Salesrpt  Printer Oflind(Endofpage);

Dcl-s Endofpage Ind Inz(*On);
Dcl-s L1Slspers     Like(Slsperson);
Dcl-s L2Dept        Like(Dept);

// ************************************************************
Read Salesfile;
L1Slspers = Slsperson;
L2Dept = Dept;
Dow Not %Eof(Salesfile);
  Select;
    When L2Dept <> Dept;
      Exsr L2break;
    When L1Slspers <> Slsperson;
      Exsr L1break;
  Endsl;
  Exsr Detailproc;
  Read Salesfile;
Enddo;
```

```
Exsr Terminate;
*Inlr = *On;
Return;



// ***************************************************************
//  Subroutine Detailproc ... done for each input record
Begsr Detailproc;
  Exsr Overflow;
  Write Detail;
  L1Amount += Saleamount;       // Roll up amount
Endsr;



// ***************************************************************
//  Subroutine L1Break ... done when salesperson changes
Begsr L1Break;
  Exsr Overflow;
  Write L1Subtot;
  L2Amount += L1Amount;         // Roll up amount
  L1Amount = 0;                 // Reset subtotal
  L1Slspers = Slsperson;        // Reset control field
Endsr;
```

```
// ***************************************************
//  Subroutine L2Break ... done when department changes
Begsr L2Break;
  Exsr L1Break;                // Process previous level break
  Exsr Overflow;
  Write L2Subtot;
  LRAmount += L2Amount;        // Roll up amount
  L2Amount = 0;                // Reset subtotal
  L2Dept = Dept;               // Reset control field
Endsr;

// ***************************************************
//  Subroutine Terminate ... done at end-of-file
Begsr Terminate;
  Exsr L2Break;                // Process final level break
  Exsr Overflow;
  Write LRTotal;
Endsr;

// ***************************************************
//  Subroutine Overflow ... print headings when necessary
Begsr Overflow;
  If Endofpage;
    Write Headings;
    Endofpage = *Off;
  Endif;
Endsr;
```

Legacy Code – Glance through this section
Legacy Code: Do

- Do operation is fixed-format variation of For operation to control count controlled loop
  - Block ends with Enddo

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C     1             Do        100           Number
C                   Add       Number        Sum
C                   Enddo
  // Same as
  //   For Number = 1 To 100;
  //     Sum += Number;
  //   Endfor;
```

Legacy Code:
Fixed Format Relational Comparisons

- Fixed format relational operators compare Factor 1 and Factor 2
  - GT (Factor 1 > Factor 2)
  - GE (Factor 1 >= Factor 2)
  - EQ (Factor 1 = Factor 2)
  - LE (Factor 1 <= Factor 2)
  - LT (Factor 1 < Factor 2)
  - NE (Factor 1 <> Factor 2)
- Used with fixed format operations
  - IF*xx*
  - WHEN*xx* (Used with SELECT)
  - DOW*xx* (Do while)
  - DOU*xx* (Do until)

Legacy Code:
Fixed Format Relational Comparisons

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C     Sales         Ifle      5000
C                   Z-add     .005          Rate
C                   Else
C     Sales         Ifle      10000
C                   Z-add     .0075         Rate
C                   Else
C     Sales         Ifle      20000
C                   Z-add     .01           Rate
C                   Else
C                   Z-add     .015          Rate
C                   Endif
C                   Endif
C                   Endif
```

Legacy Code:
Fixed Format Relational Comparisons

- Can combine comparisons using Andxx or Orxx operations

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C     Age           Ifge      65
C     Status        Andeq     'R'
C                   Add       1             Count
C                   Endif
```

Legacy Code: Conditioning Indicators

- Conditioning indicators execute or bypass single line of code based upon indicator's state
  - Precede indicator with N to test for *Off state

```
*.. 1 ...+... 2 ...+... 3 ...+... 4 ...+... 5 ...+... 6 ...+... 7 ...+... 8
CL0N01Factor1+++++++Opcode(E)+Factor2+++++++Result++++++++Len++D+HiLoEq....

C                   Z-add     .10           Taxrate
C   50              Z-add     .18           Taxrate
C   51              Z-add     .25           Taxrate
C   52              Z-add     .31           Taxrate
C  N53              Add       .05           Taxrate
```